



Technische
Universität
Braunschweig

**Decision
Support**

Institut für Wirtschaftsinformatik



Data Driven Decision Making

Lecture 5 – Markov Decision Process

Agenda

- **Markov decision process**
- Optimal solutions
- Curses of dimensionality
- Modeling discussion

Operations Research

- Business Problems



$$\min \sum_{i=1}^m \sum_{j=1}^n c_{ij} x_{ij}$$

$$\text{u.d.N.} \quad \sum_{j=1}^n x_{ij} = a_i \quad (i = 1, \dots, m)$$

$$\sum_{i=1}^m x_{ij} = b_j \quad (j = 1, \dots, n)$$

$$x_{ij} \geq 0 \quad (i = 1, \dots, m; j = 1, \dots, n)$$

- Mathematical Modeling



- Solution Methods (Heuristics)

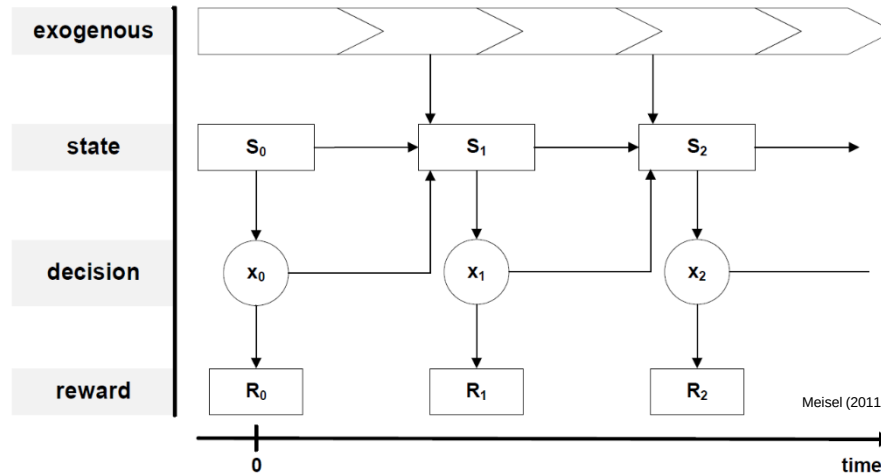


Modeling

- Model connects real-world application with solution methodology
- Modeling translates verbal descriptions into mathematical notation
- A model provides an unambiguous definition of the problem
- Different problems may have the same model
- Different models can define the same problem
- A specific modeling can support solution methodology

Dynamic Decision Process

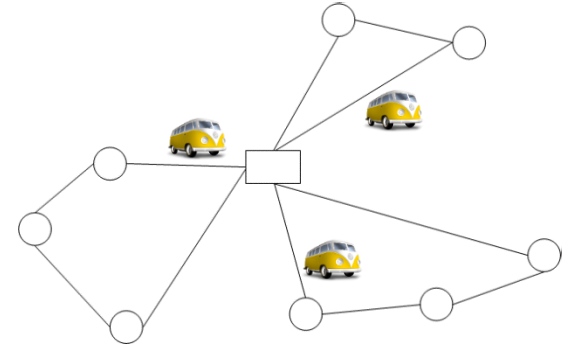
- Two processes:
 - Exogeneous: Changing information (stochasticity)
 - Endogenous: Adaption of plans (dynamism)



→ A model should capture stochasticity and dynamism (over time)

“Standard” Static Vehicle Routing Problem

- Depot, customers, vehicles
- Constraint: capacity
- Objective: minimize travel time
- Decisions
 - Assignment of customers to vehicles
 - Sequence of customers per vehicle
- Modeling: mixed-integer programming
- Solution: set of routes
- No consideration of temporal developments
- No consideration of stochasticity



minimize

$$\sum_{(i,j) \in A} \sum_{k \in V} c_{ij} z_{ijk}$$

subject to

$$\sum_{k \in V} y_{ik} \geq 1 \quad i \in C$$

$$\sum_{(i,j) \in A} z_{ijk} = y_{ik} \quad i \in N, k \in V$$

$$\sum_{(j,i) \in A} z_{jik} = y_{ik} \quad i \in N, k \in V$$

$$\sum_{(j,i) \in A} x_{jik} - \sum_{(i,j) \in A} x_{ijk} = d_i y_{ik} \quad i \in C, k \in V$$

$$x_{ijk} \leq L z_{ijk} \quad (i,j) \in A, k \in V$$

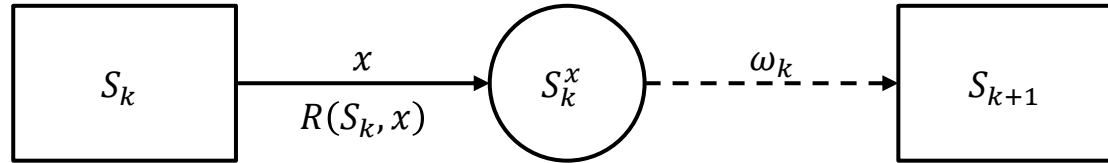
$$y_{1k} = 1 \quad k \in V$$

$$x_{ijk} \geq 0 \quad (i,j) \in A, k \in V$$

$$y_{ik} \in \{0, 1\} \quad i \in N, k \in V$$

$$z_{ijk} \in \{0, 1\} \quad (i,j) \in A, k \in V$$

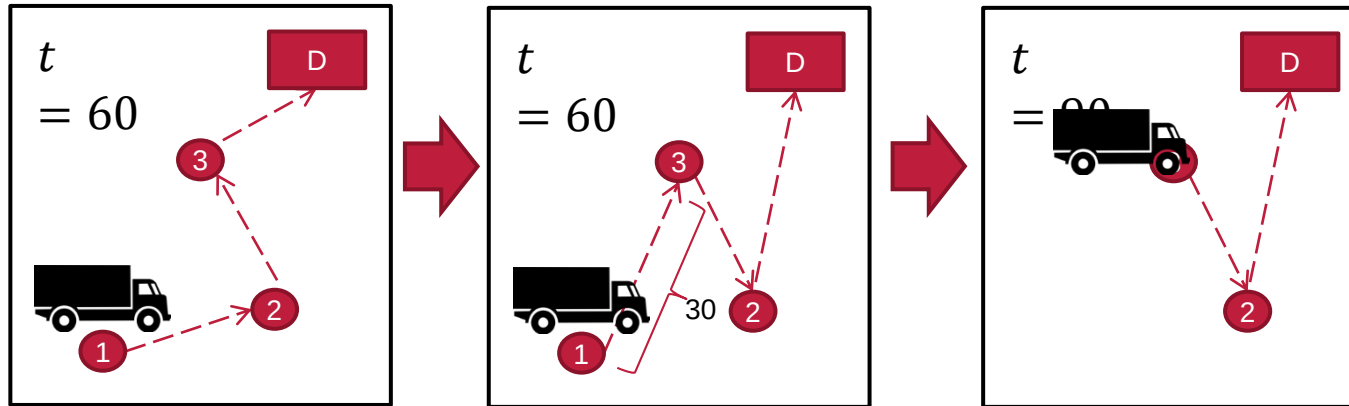
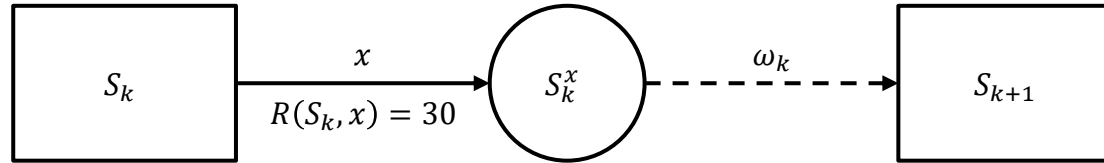
Markov Decision Process



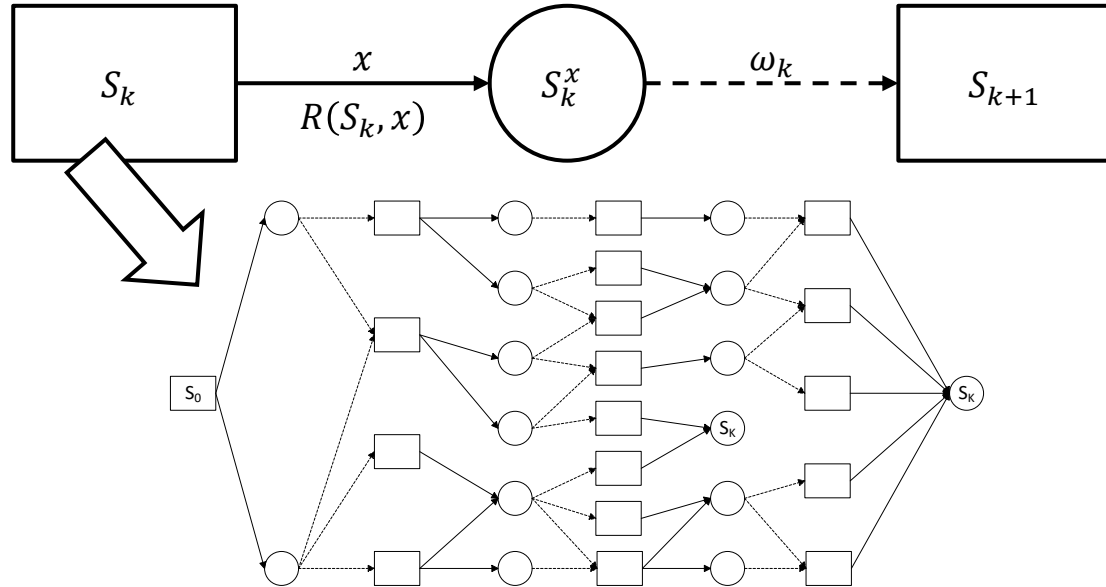
Components:

- Decision point k (either periodic or event triggered)
- Decision state $S_k \in \mathbf{S}$
- Decisions $x \in X$ (e.g., with Mixed-Integer Programming)
- Reward $R(S_k, x) \in \mathbb{R}$ (or costs as negative reward)
- Post-decision state $S_k^x \in \mathbf{S} \times X$
- Stochastic transition $\omega_k \in \mathbf{S} \times X \times \mathbf{S}$ (e.g., with probabilities $\mathbb{P}: \mathbf{S} \times X \times \mathbf{S} \rightarrow [0, 1]$)

Example: Parcel Service



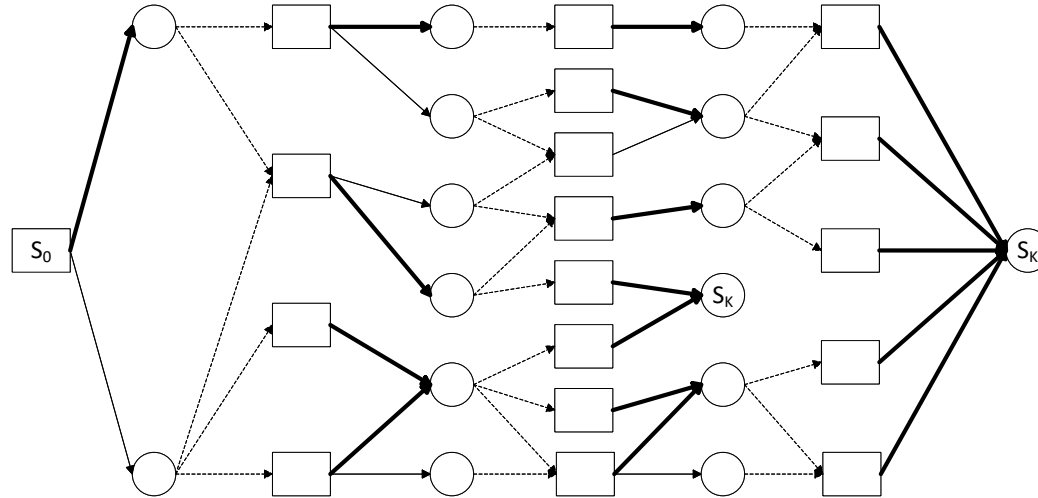
Problem Modeling: Markov Decision Process



- Initial state S_0
- Final state S_K

Solution: Decision Policy

- A decision policy assigns a decision to every state



- Objective: find optimal policy π^* maximizing expected overall rewards

Optimal Solution

- An optimal solution π^* satisfies:

$$\pi^* = \arg \max_{\pi \in \Pi} \mathbb{E} \left[\sum_{i=0}^K R(S_i, X_i^{\pi^*}(S_i)) | S_0 \right]$$

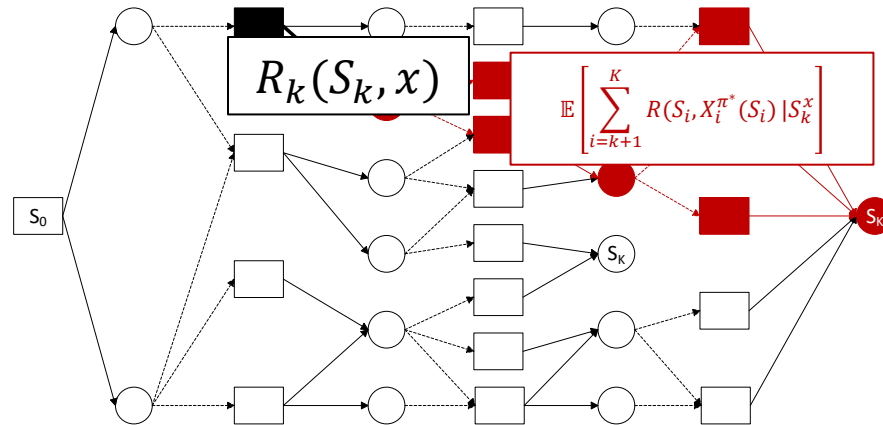
- Policy π^* represents the policy $\pi \in \Pi$, maximizing the expected reward
- Given initial state S_0
- $X_k^{\pi}(S_k)$ indicates the decision made by policy π in state S_k
- For minimization, we search for $\arg \min$

Bellman Equation

- An optimal policy satisfies:

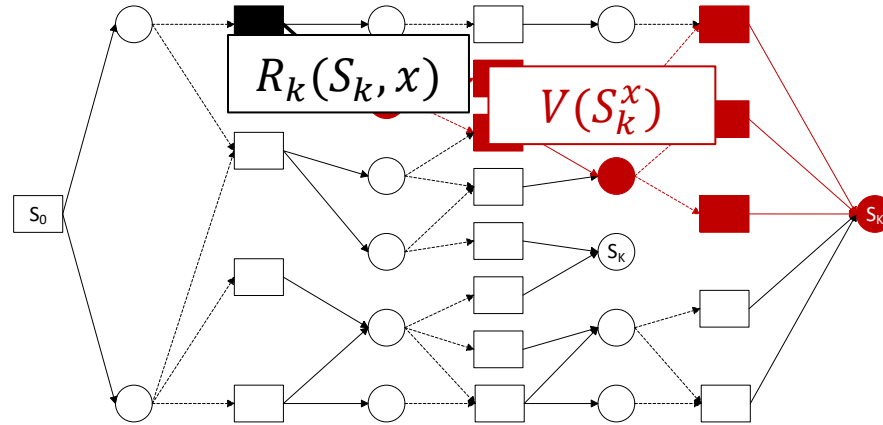
$$X_k^{\pi^*}(S_k) = \arg \max_{x \in X(S_k)} \left(R(S_k, x) + \mathbb{E} \left[\sum_{i=k+1}^K R(S_i, X_i^{\pi^*}(S_i) | S_k^x) \right] \right)$$

- Maximize immediate plus expected future reward



Value Function

- Expected future reward is called the **value** $V(S_k^x)$ of post-decision state S_k^x
- The value function maps a (post-decision) state to the expected future reward

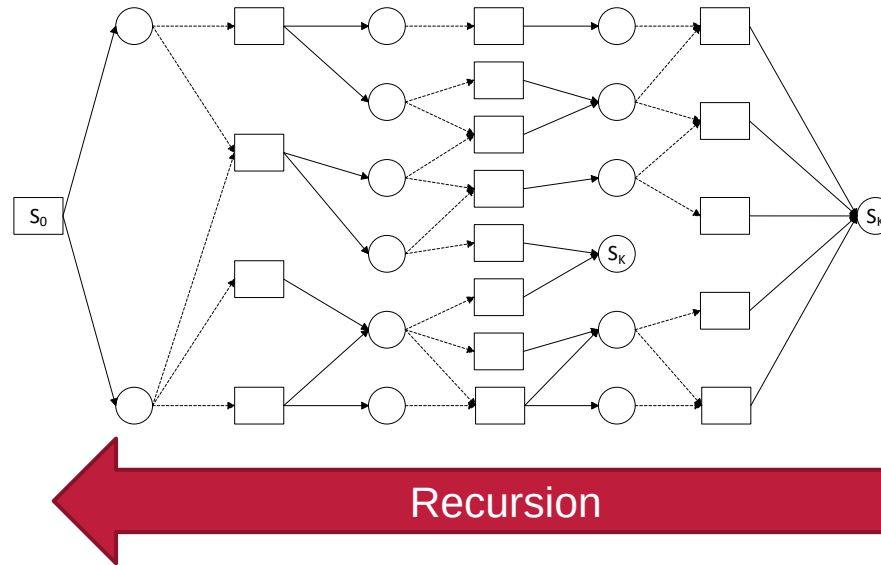


Value Function

- The value function for the optimal policy satisfies:
 - $V(S_k) = \max R(S_k, x) + V(S_k^x)$ (deterministic decision)
 - $V(S_k^x) = \sum_{S_{k+1}} \mathbb{P}(S_{k+1} = (S_k^x, \omega_k)) V(S_{k+1})$ (stochastic transition)
- Values can be calculated recursively for a finite horizon
 - $V(S_K) = 0$ (value of final state is 0)
 - $V(S_{K-1}^x) = \sum_{S_K} \mathbb{P}(S_K = (S_{K-1}^x, \omega_{K-1})) V(S_K)$ (last but one post-decision state)
 - $V(S_{K-1}) = \max R(S_{K-1}, x) + V(S_{K-1}^x)$ (last but one state)
 - $V(S_{K-2}^x) = \sum_{S_{K-1}} \mathbb{P}(S_{K-1} = (S_{K-2}^x, \omega_{K-2})) V(S_{K-1})$ (last but two post-decision state)
 - $V(S_{K-2}) = \max R(S_{K-2}, x) + V(S_{K-2}^x)$ (last but two state)
 -

Values can be calculated recursively

- $V(S_K) = 0$
- $V(S_k) = \max R(S_k, x) + V(S_k^x)$
- $V(S_k^x) = \sum_{S_{k+1}} \mathbb{P}(S_{k+1} = (S_k^x, \omega_k)) V(S_{k+1})$



Agenda

- Markov Decision Process
- **Optimal Solutions**
- Curses of Dimensionality
- Modeling discussion

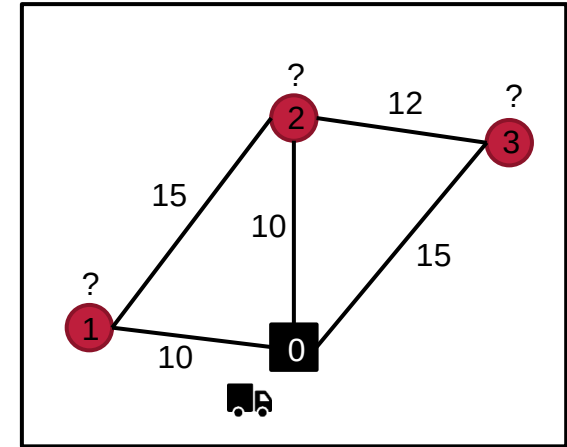
Example

- Illustrate Markov Decision Process
- Show recursive dynamic programming solution
- Indicate Curses of Dimensionality



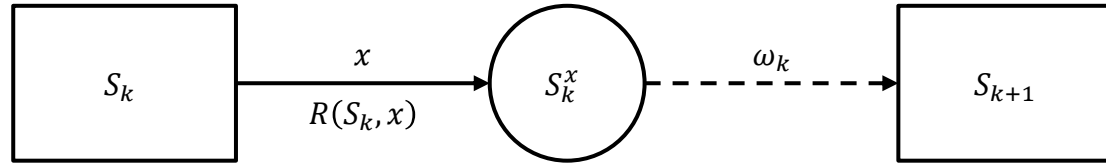
Example

- Serve 3 customers in predefined sequence (1-2-3)
- Stochastic demand
 - Either 2 or 1 unit(s)
 - Same probability (50%)
- Vehicle with capacity 4
- Replenishment at the depot
- If demand is larger than fill level:
serve partial demand → replenishment → serve remaining demand



- When should we return to the depot for replenishment to minimize expected travel time?
- What are the expected costs?

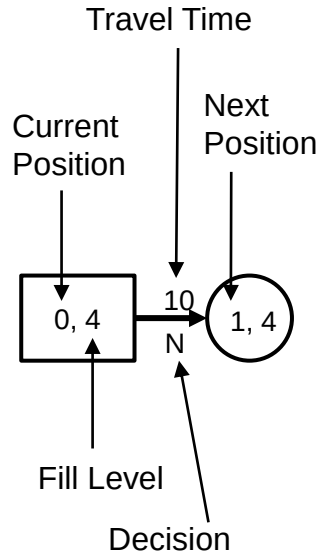
Markov Decision Process



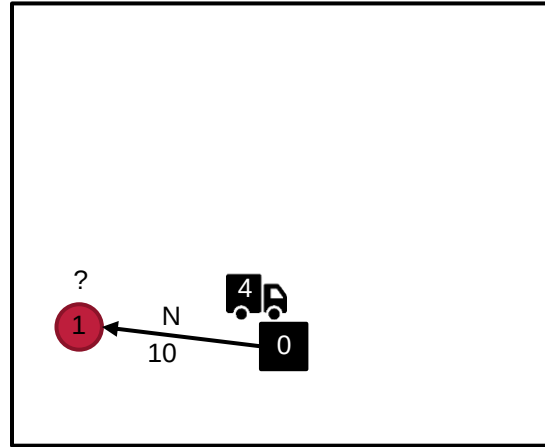
Components:

- Decision point k : Arrival at customer
- State S_k : Position & Fill level vehicle
- Decision x : Replenish at depot (A) or next customer (N)
- Cost (Negative Reward) $R(S_k, x)$: Travel time
- Post-Decision State S_k^x : Position & Fill level vehicle
- Stochastic Transition ω_k : Demand Realization

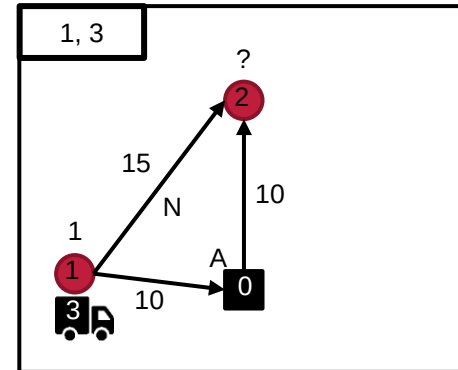
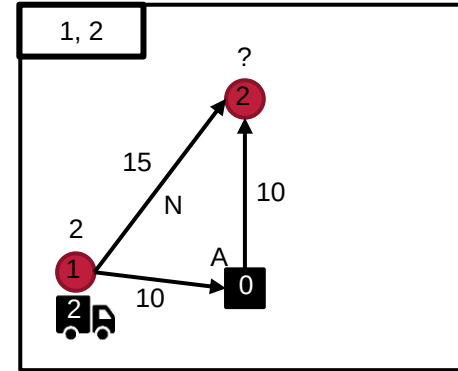
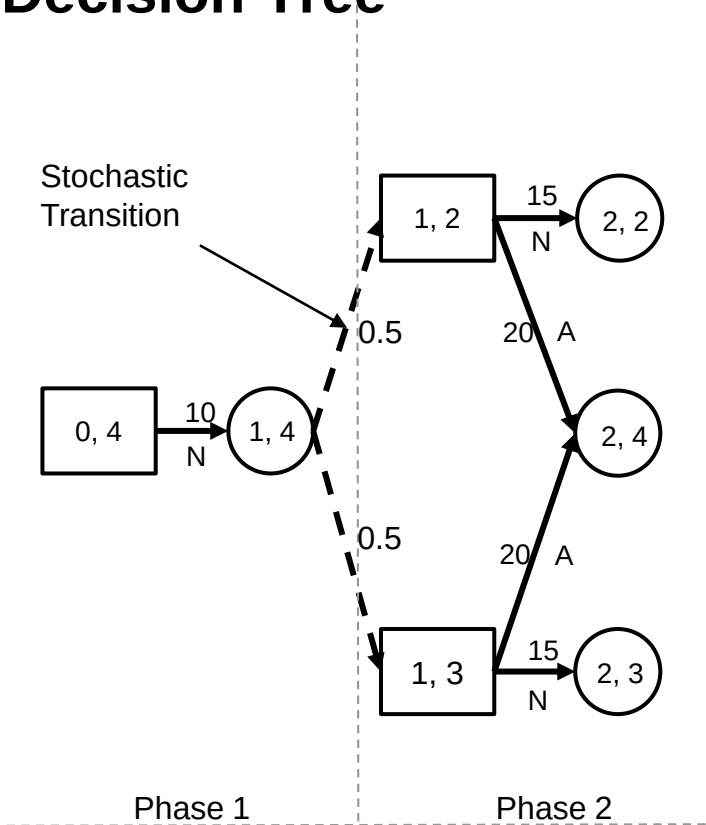
MDP-Decision Tree



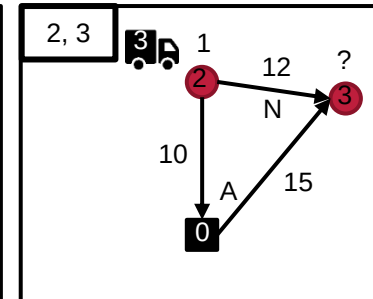
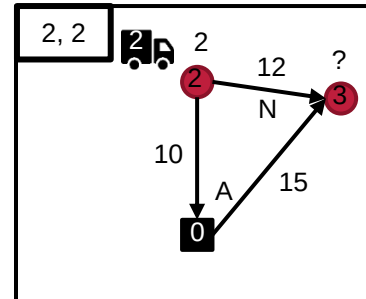
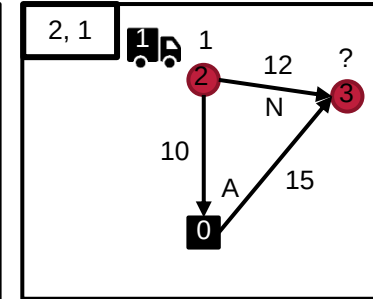
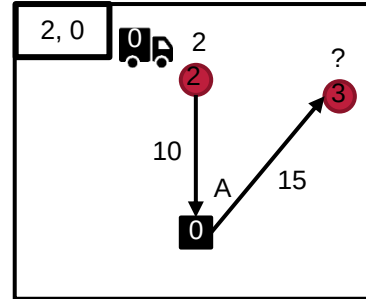
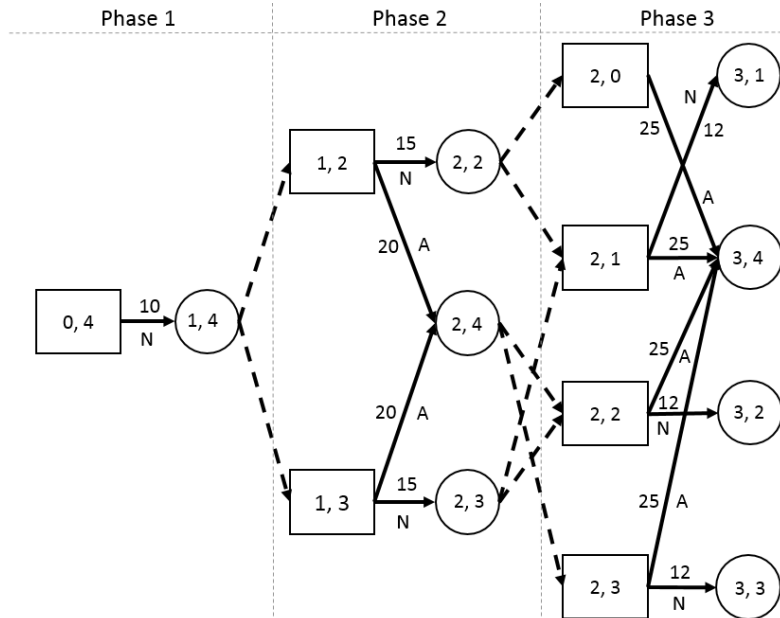
Phase 1



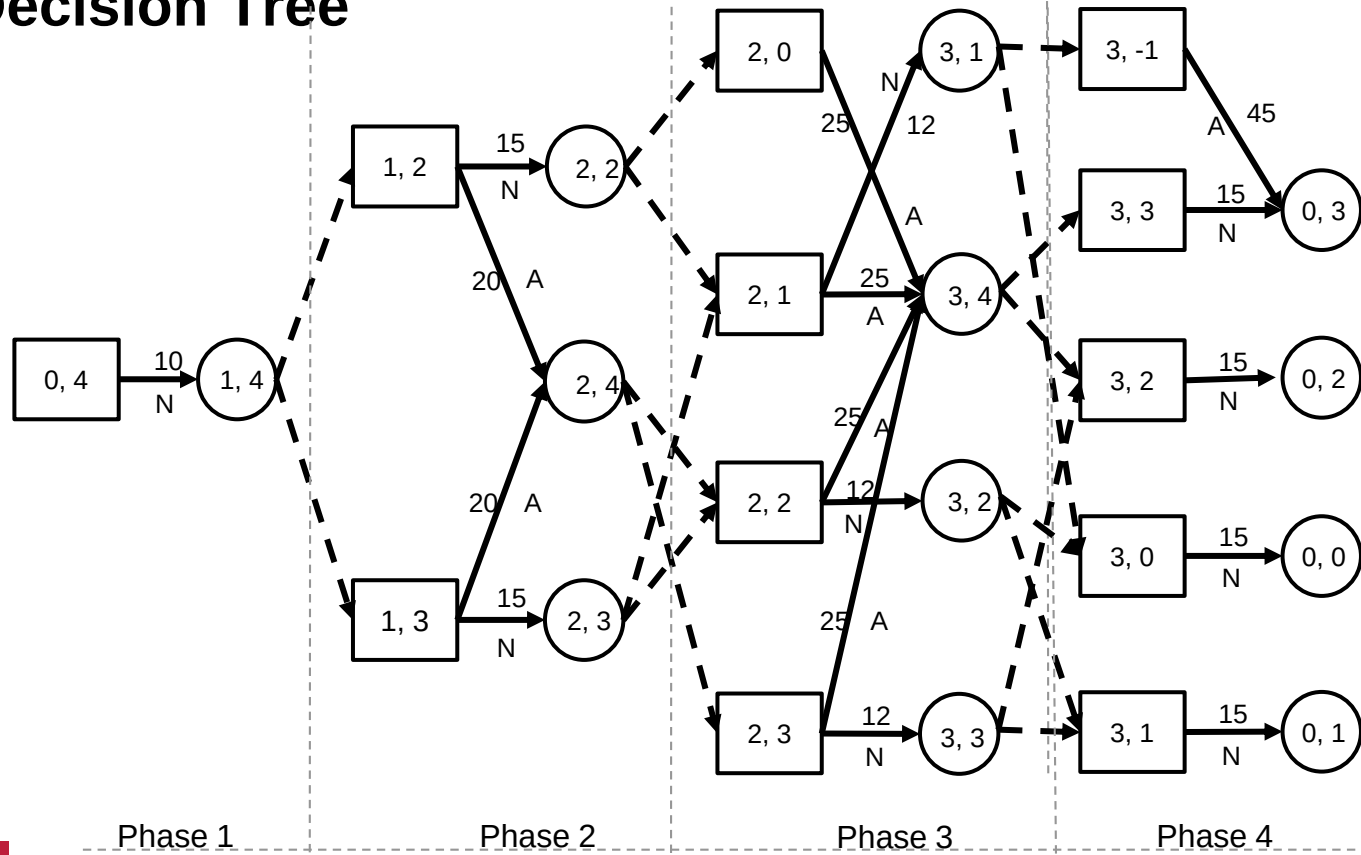
MDP-Decision Tree



MDP-Decision Tree

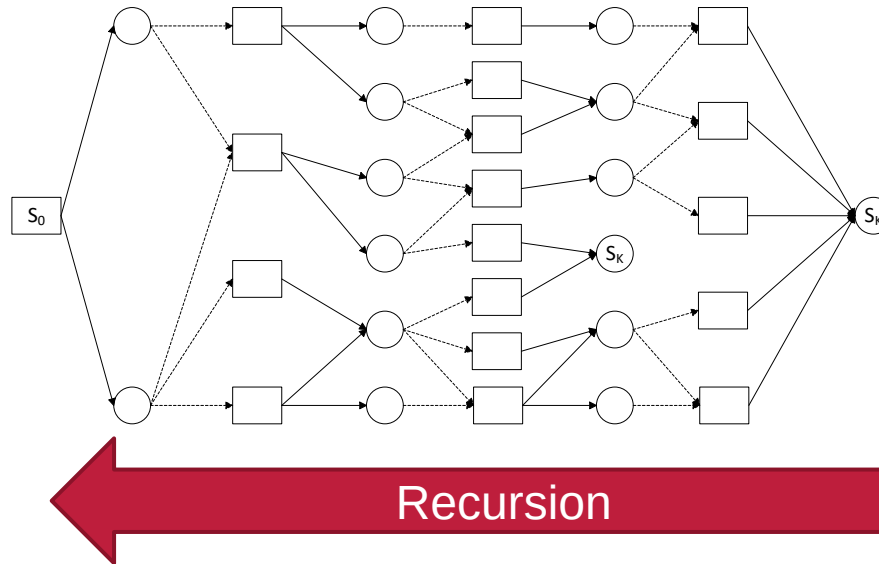


MDP-Decision Tree



Values can be calculated recursively

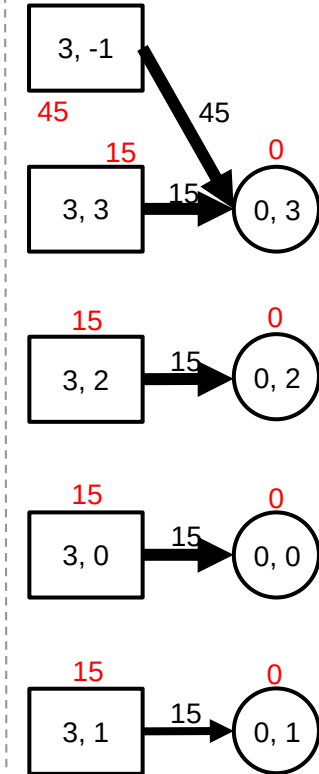
- $V(S_K) = 0$
- $V(S_k) = \max R(S_k, x) + V(S_k^x)$
- $V(S_k^x) = \sum_{S_{k+1}} \mathbb{P}(S_{k+1} = (S_k^x, \omega_k)) V(S_{k+1})$



Calculating Optimal Policy

Phase k	State S_k	Decision x	Next State S_{k+1}	Value of Decision $R_k(S_k, x) + V(S_k^*)$	Optimal Value $V^*(S_k)$	Optimal Decision x^*
4	3, -1	A	-	45 + 0	45	A
	3, 0	N	-	15 + 0	15	N
	3, 1	N	-	15 + 0	15	N
	3, 2	N	-	15 + 0	15	N
	3, 3	N	-	15 + 0	15	N

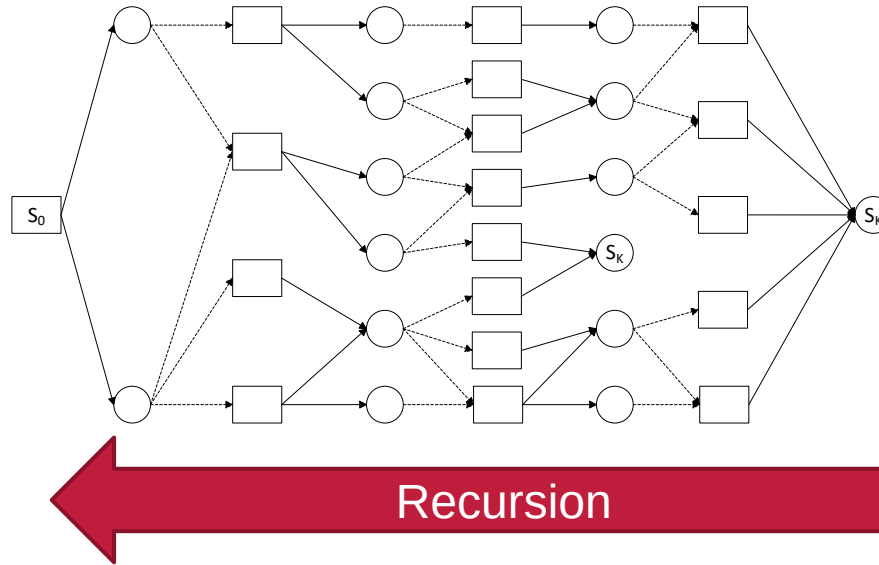
MDP-Decision Tree



Phase 4

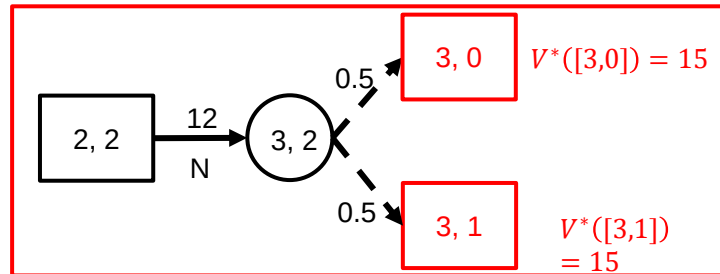
Values can be calculated recursively

- $V(S_K) = 0$
- $V(S_k) = \max R(S_k, x) + V(S_k^x)$
- $V(S_k^x) = \sum_{S_{k+1}} \mathbb{P}(S_{k+1} = (S_k^x, \omega_k)) V(S_{k+1})$



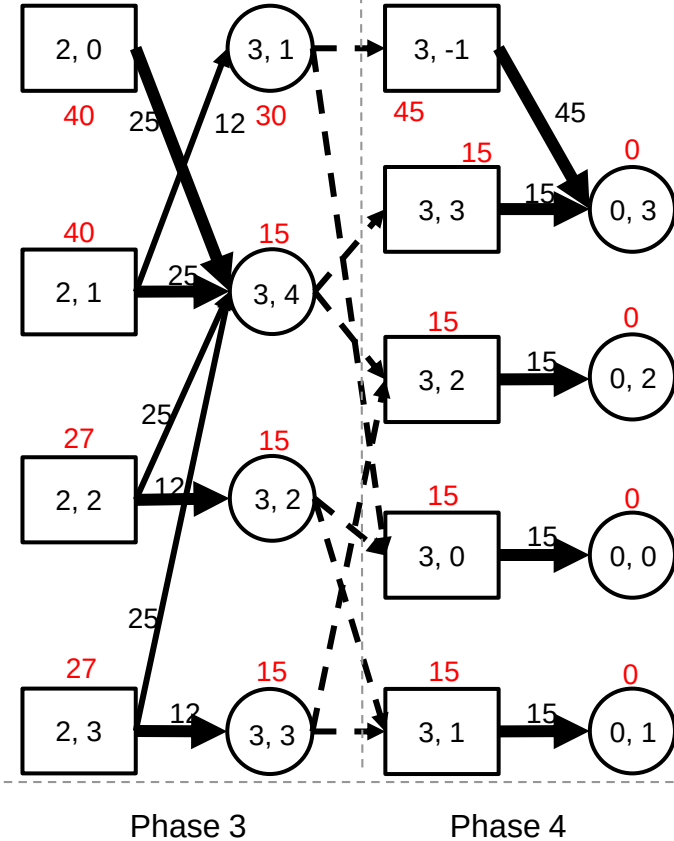
Calculating Optimal Policy

Phase k	State S_k	Decision x	Next State S_{k+1}	Value of Decision $R_k(S_k, x) + V(S_k^x)$	Optimal Value $V^*(S_k)$	Optimal Decision x^*
3	2, 0	A	[3,2] , [3,3]	$25 + (15*0.5 + 15*0.5) = 40$	40	A
	2, 1	N	[3,-1], [3,0]	$12 + (45*0.5 + 15*0.5) = 42$	40	A
		A	[3,2], [3,3]	$25 + (15*0.5 + 15*0.5) = 40$		
	2, 2	N	[3,0], [3,1]	$12 + (15*0.5 + 15*0.5) = 27$	27	N
		A	[3,2], [3,3]	$25 + (15*0.5 + 15*0.5) = 40$		
	2, 3	N	[3,2], [3,1]	$12 + (15*0.5 + 15*0.5) = 27$	27	N
		A	[3,2], [3,3]	$25 + (15*0.5 + 15*0.5) = 40$		



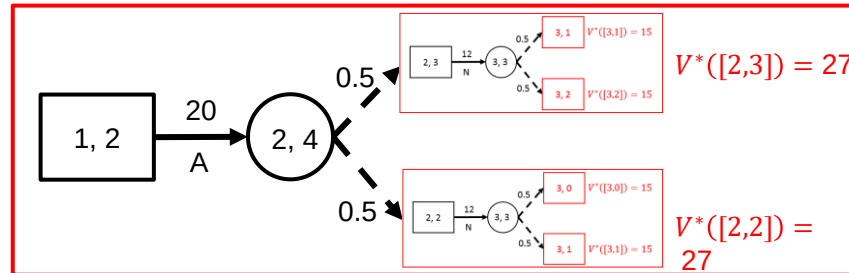
$$V^*([2, 2]) = 27$$

MDP-Decision Tree



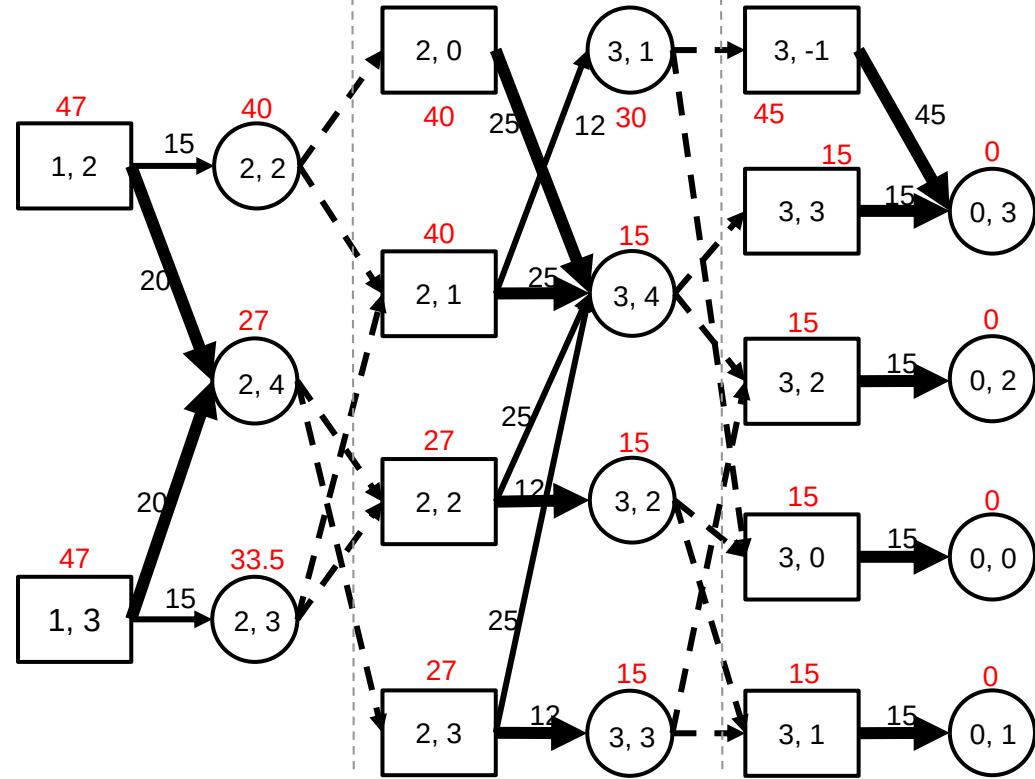
Calculating Optimal Policy

Phase k	State S_k	Decision x	Next State S_{k+1}	Value of Decision $R_k(S_k, x) + V(S_k^x)$	Optimal Value $V^*(S_k)$	Optimal Decision x^*
2	1, 2	N	[2,0], [2,1]	$15 + (40*0.5 + 40*0.5) = 55$	47	A
		A	[2,2], [2,3]	$20 + (27*0.5 + 27*0.5) = 47$		
	1, 3	N	[2,1], [2,2]	$15 + (40*0.5 + 27*0.5) = 48.5$	47	A
		A	[2,2], [2,3]	$20 + (27*0.5 + 27*0.5) = 47$		



$$V^*([1, 2]) = 47$$

MDP-Decision Tree



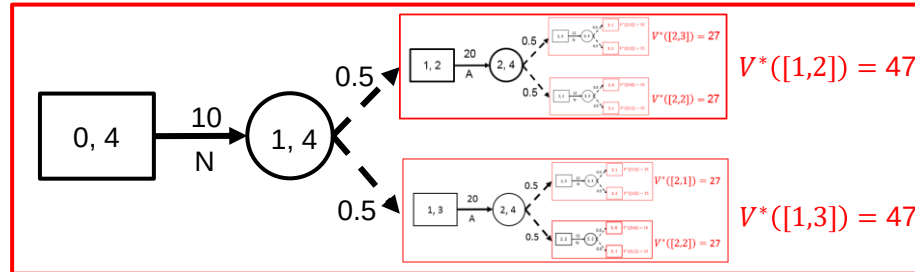
Phase 2

Phase 3

Phase 4

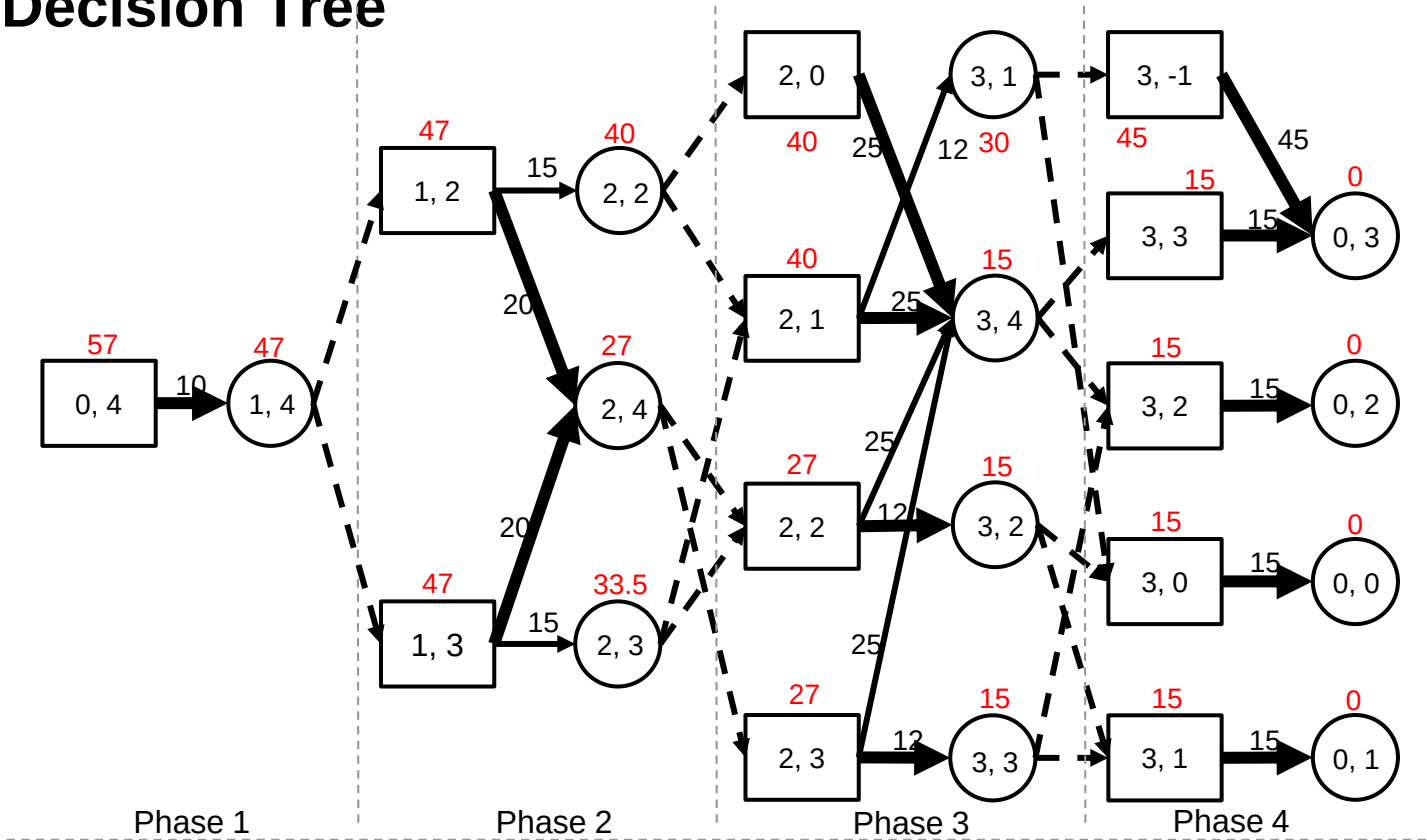
Calculating Optimal Policy

Phase k	State S_k	Decision x	Next State S_{k+1}	Value of Decision $R_k(S_k, x) + V(S_k^x)$	Optimal Value $V^*(S_k)$	Optimal Decision x^*
1	0, 4	N	[1,2] , [1,3]	$10 + (47 \cdot 0.5 + 47 \cdot 0.5) = 57$	57	N



$$V^*([0, 4]) = 57$$

MDP-Decision Tree



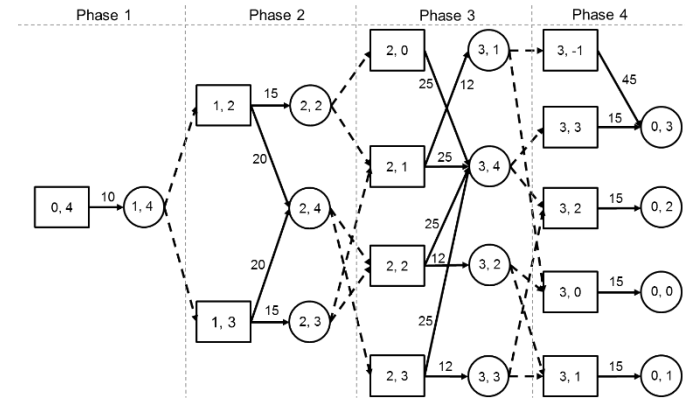
Conclusion I: Management

- Expected travel time is 57 minutes
- Vehicle always returns after serving first customer



Conclusion II: Calculation

- Calculation possible
- Problem with
 - 4 Decision points
 - 12 States
 - 17 Decisions
 - 12 Post-decisions states
 - 16 Transitions
 - 16 Calculations



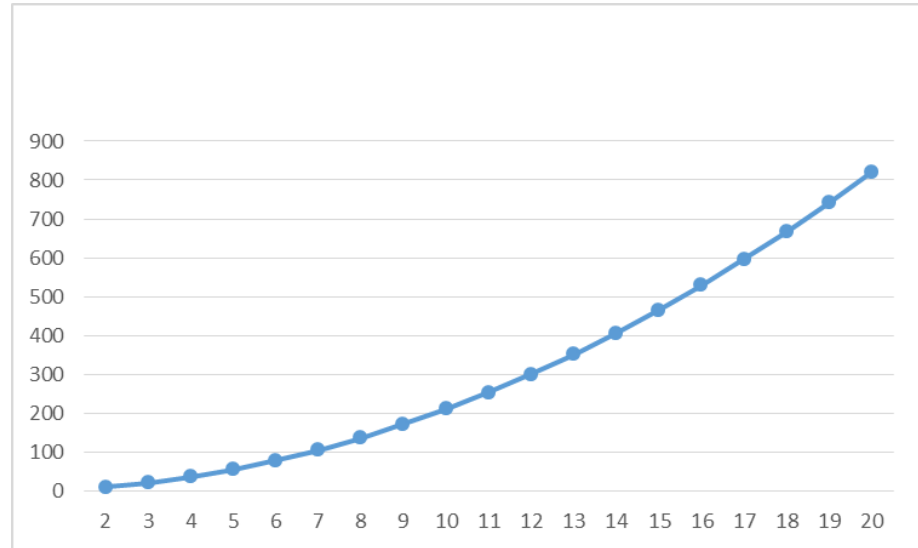
- What if we increase the dimensions? Curses of Dimensionality

Agenda

- Markov Decision Process
- Optimal Solutions
- **Curses of Dimensionality**
- Modeling discussion

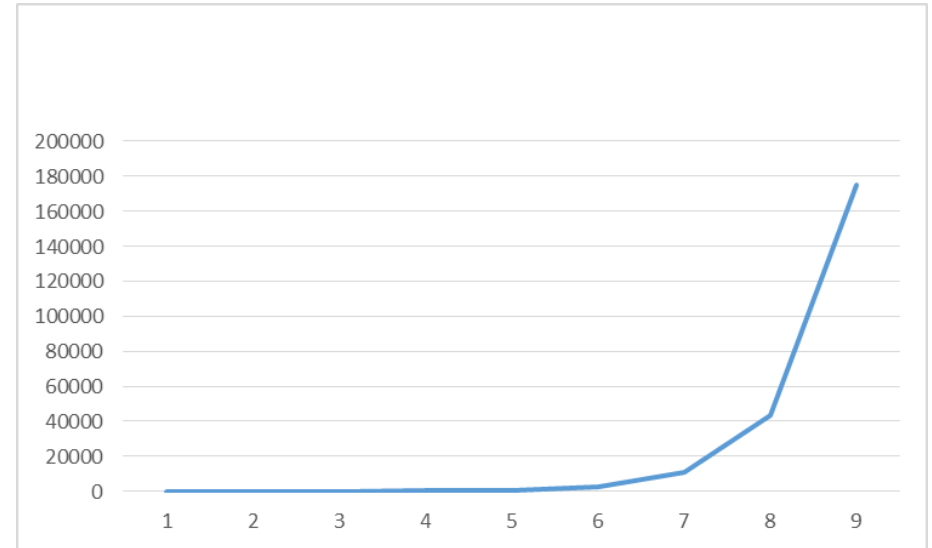
Transition/Information Space

- Stepwise increase of potential demand options
- 2,3,...,20,
- Up to 900 states with 20 options



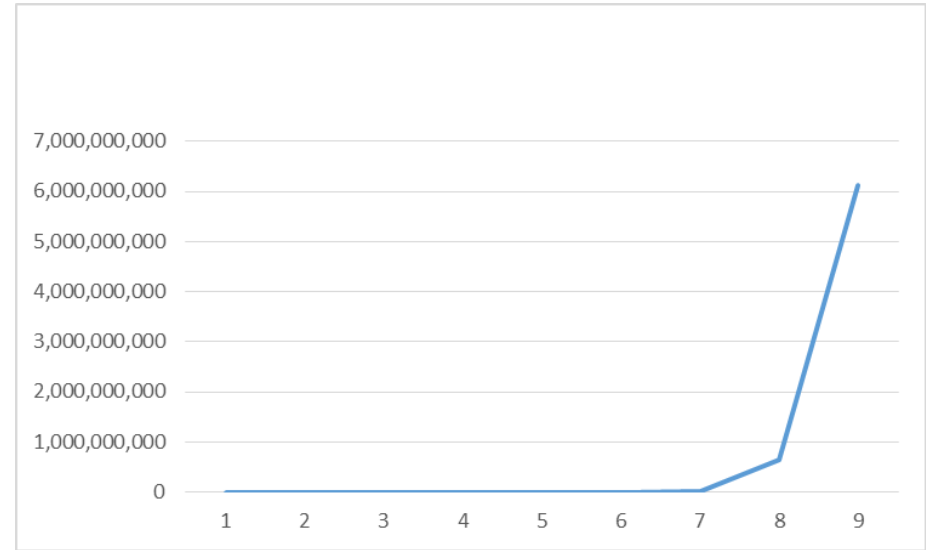
Number of Customers

- Stepwise increase in number of customer
- 1,...,9 customers
- Up to 200,000 states for problem with 9 customers



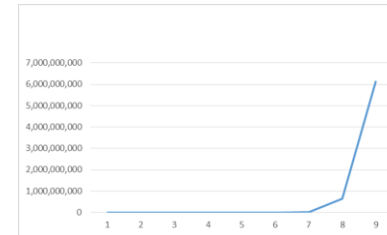
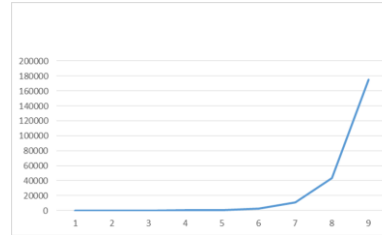
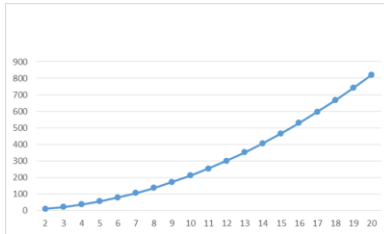
Increase Customer Number and Integrate Routing Decisions

- Stepwise increase in number of customer
- 1,...,9 customers
- Up to 6 billion states for problem with 9 customers



Curses of Dimensionality

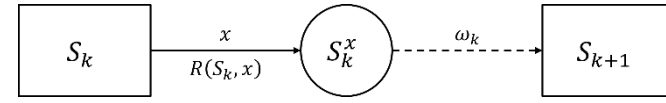
- State Space
- Decision Space
- Information Space
- With increasing dimensionality, recursion becomes infeasible.
- For (nearly) all real-world problems, an optimal solution cannot be found



Agenda

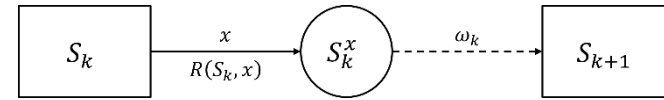
- Markov Decision Process
- Optimal Solutions
- Curses of Dimensionality
- **Modeling discussion**

Decision Point



- Decision points define when a decision is made.
- Notation: $k = 0, \dots, K$
- Not necessarily equidistant and known
- Examples:
 - Every day, every hour, etc.
 - Whenever a vehicle served a customer/ a job is produced
 - Whenever a new customer requests/ new job comes in
 - Self-Induced (“I will wait 5 minutes and then decide”)
- The time a decision is made depends on its “importance” to the stakeholders
- Tradeoff between fast reaction and information gain.

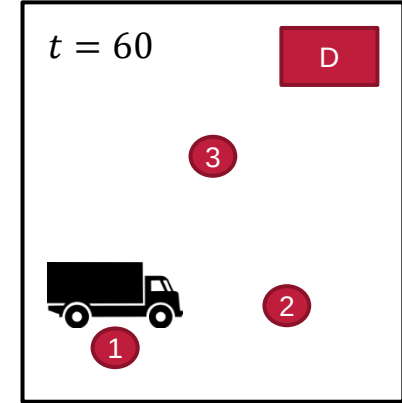
State



- A state contains all information available to make a decision
- Notation: S_k in decision point k
- Information comprises
 - Environment: time, traffic, weather, etc.
 - Resources: vehicle locations, machine availability times, etc.
 - Demand: customers and locations, set of production jobs, etc.
- Resources and demand may have additional attributes such as capacity, deadlines, etc.

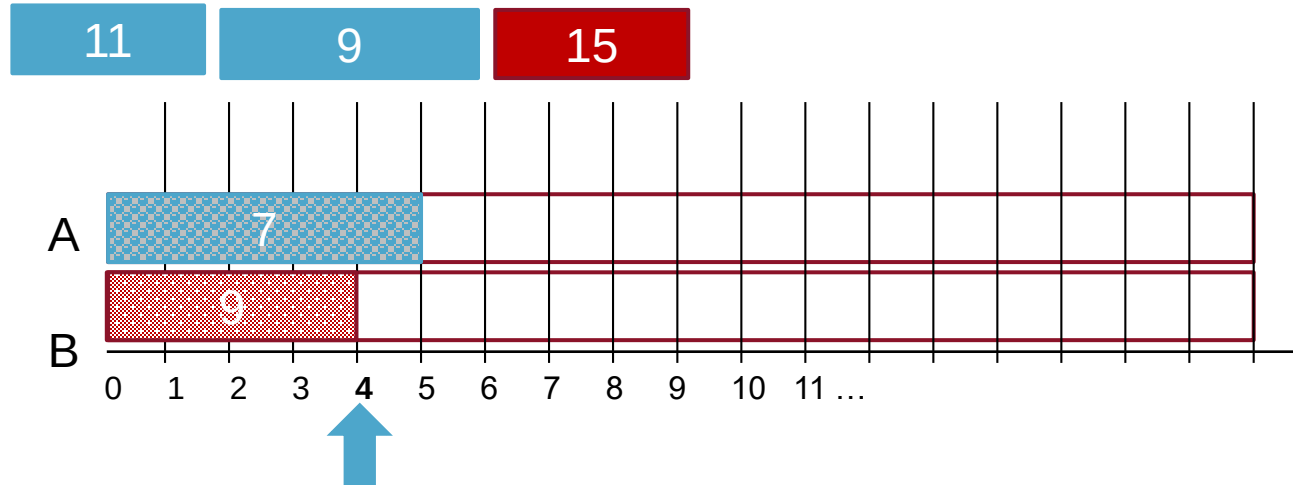
Example 1: Delivery Routing

- Environment: $t = 60$, travel time matrix
- Resources: vehicle location
- Demand: Set of customers still to serve



Example 2: Line Production

- Environment: $t = 4$
- Resources: availability times of the machines plus previous job: (A, 5, blue), (B, 4, red)
- Demand: Set of jobs with information (Type, Duration, Deadline):
 - (blue,3,11), (blue,4,9), (red, 3, 15)



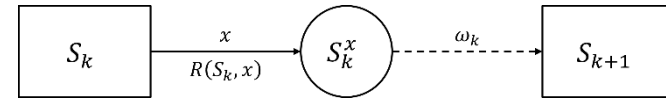
Example 3: Inventory Management

- Environment: $t = 2$
- Resources: Current fill level
- Demand: Not realized yet

$t = 2$



Decisions and Rewards



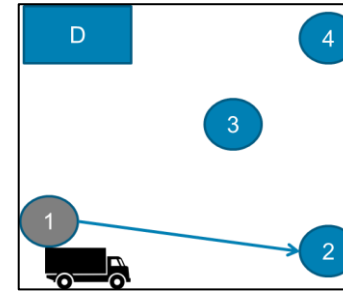
- Decisions are made about the use/spending of resources to serve demand
- Notation: $x_k \in X(S_k)$ in state S_k
- Feasibility depend on
 - Constraints in resource use (travel/set-up times, capacities, etc.)
 - Constraints in service of demand (deadlines, amount, etc.)
- Each decisions causes costs and/or rewards $R(S_k, x_k)$:
 - Costs in use of resources
 - Rewards in service of demand
- Often, either costs or rewards are moved to the constraints.
- Reward can be a random variable

Examples 1: Dynamic Vehicle Routing

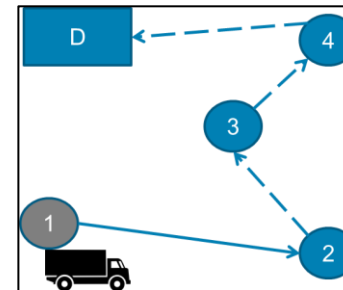
- Decisions and rewards can be modeled either as immediate action or as plan updates.
- Immediate action is the least commitment necessary (next customer to serve, next job to produce)
- Plan updates consider a sequence of tentative actions where only the first is implemented
- Each model type requires different reward functions (reward of action vs. marginal change)
- Opinion: Plan updates are more intuitive and connect better to methodology

Ulmer et al. 2020: On Modeling Stochastic and Dynamic Vehicle Routing Problems

Action:

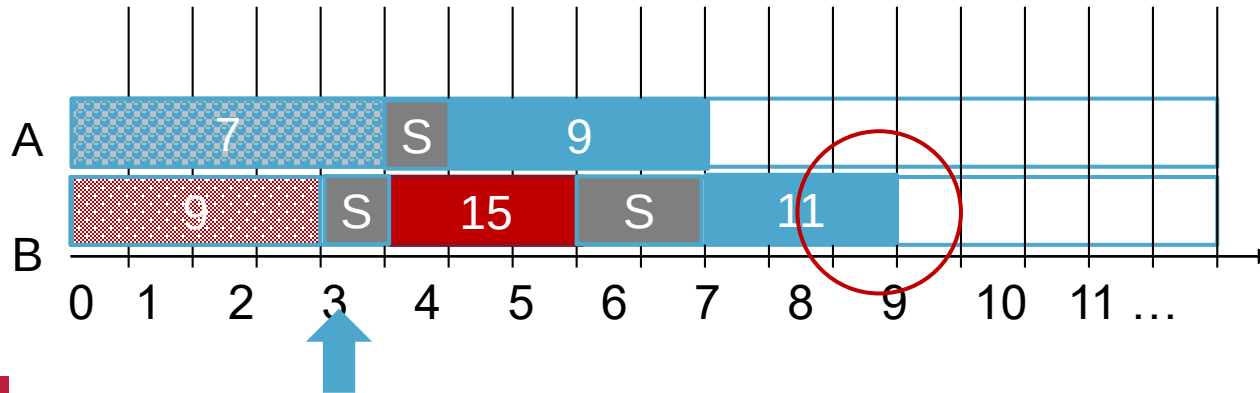


Plan:



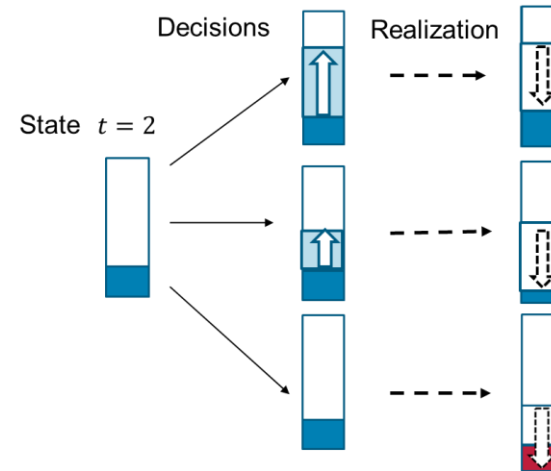
Example 2: Connecting Decisions to Methods

- Without plan, we just decide about the next job on line B
- With plan, we tentatively plan all jobs and only implement the next job on line B
- A solution method aims on minimizing delay
- Operating on a set of plans indicates impact on delay for all jobs
- Only operating on current action seems unintuitive
- Difference in reward/delay: current action: 0, plan: 1



Example 3: Uncertain Reward

- Inventory management problem has two types of costs:
 - Inventory holding costs (known at time of decision)
 - Emergency replenishment costs:
 - Stochastic
 - Unknown at time of decision
- Also very common when
 - Customers can choose products
 - Travel times are stochastic
 - Drivers can reject offers
 - Etc.



Post-Decision State

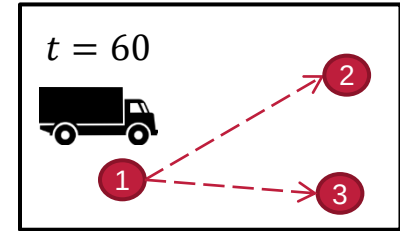
- Combination of state S_k and decision x_k leads to post-decision state S_k^x
- Deterministic part of the overall transition from state to state.
- Often contains similar information as the state with changes (e.g., no open requests/jobs)
- Can be seen as the “current plan.”
- Explicit modeling supports
 - Communication (We see the implementation of the decision)
 - Methodology (compare Bellman Equation, $\max R(S_k, x) + V(S_k^x)$)

- Combination of state S_k and decision x_k leads to post-decision state S_k^x
- Deterministic part of the overall transition from state to state.
- Often contains similar information as the state with changes (e.g., no open requests/jobs)
- Can be seen as the “current plan.”
- Explicit modeling supports
 - Communication (We see the implementation of the decision)
 - Methodology (compare Bellman Equation, $\max R(S_k, x) + V(S_k^x)$)

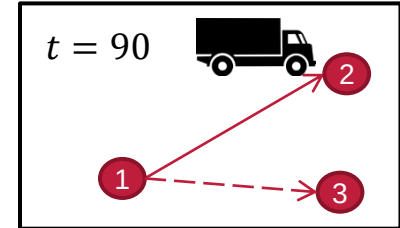
Example: Decision Time of Post-Decision State

- Post-decision state may have the same or a later decision time as the corresponding state
- Example 1
 - Deterministic travel times, but stochastic demand at new location
→ later decision time
- Example 2
 - Deterministic demand at new location, but stochastic travel times
→ same decision time

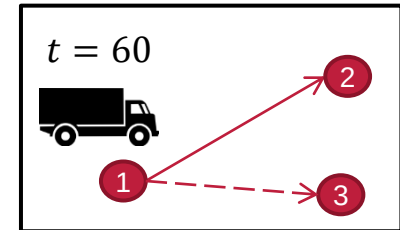
Example state and possible decisions:



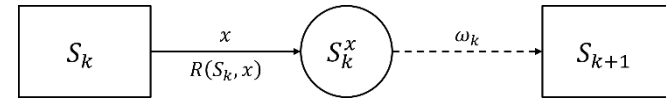
Post-decision state for example 1:



Post-decision state for example 2:



Stochastic Information and Transitions



- Observation of stochastic information ω_k given post-decision state S_k^x leads to new state S_{k+1} .
- Stochastic part of the overall transition from state to state.
- Each stochastic information has a probability (in theory).
- If transition is not self-evident, sometimes transition function is explicitly modeled (compare Powell 2011)
- Transitions comprise:
 - Observation of new customers/jobs/demand
 - Customer selecting products
 - Etc.

Coding MDPs

For coding MDPs, five components are required:

1. Exogeneous information, for example, lists of jobs/customers with all relevant details (duration, location, potentially choice model details, etc.)
2. Simulation loop, for example, over time or the job/customer list
3. State information as combination of previous post-decision state and new information
4. Decision policy, assigning a decision to a state (see next week) and creating reward and post-decision state
5. Variable to track observed rewards

Outlook

- Approximate Dynamic Programming
- Rolling Horizon Optimization
- Policy and Cost Function Approximation
- Simulation methods

